

Fundamentos da Programação Orientada a Objetos

PEX0130 - Programação Orientada a Objetos

Bruno Silva

Objetivo

1. Compreender os conceitos essenciais da POO, incluindo modelos, classes, objetos, encapsulamento, herança, polimorfismo e abstração, com exemplos práticos e diagramas.
2. Iniciar na programação em Java.

1. O que é um Programa de Computador?

Um programa é um conjunto de instruções que manipula **dados** para realizar tarefas específicas.

Na **POO**, dados e comportamentos são organizados em **objetos** que interagem entre si.

2. Modelos em POO

Modelos são representações simplificadas de entidades do mundo real, contendo:

- **Dados (atributos):** Características do objeto.
- **Operações (métodos):** Comportamentos ou ações.

Exemplo: Modelo de uma Lâmpada

Lampada
- boolean ligada
+ void ligar() + void desligar() + boolean estaLigada()

3. Os 4 Pilares da POO

3.1. Encapsulamento

Ocultação do estado interno de um objeto, expondo apenas uma interface controlada.

Ocultação do comportamento e dos dados relativos ao processo permite que o usuário se preocupe apenas em "usar" a classe.

Vantagens

1. Controle de acesso aos dados

- i. Atributos privados só são modificados via métodos públicos (ex: setSaldo).
- ii. Evita corrupção de estado (ex: saldo negativo não validado).

2. Manutenibilidade

- i. Alterações internas na classe não afetam outros componentes (ex: mudar de double para BigDecimal para saldo).

3. Segurança

- i. Dados sensíveis são protegidos (ex: senhas, IDs internos).

4. Acoplamento reduzido

- i. Classes não dependem da implementação interna umas das outras, apenas da interface pública.

Exemplo

```
public class ContaBancaria {  
    private double saldo; // encapsulado  
  
    public void depositar(double valor) {  
        if (valor > 0) saldo += valor;  
    }  
  
    public double getSaldo() {  
        return saldo;  
    }  
}
```


3.2. Herança

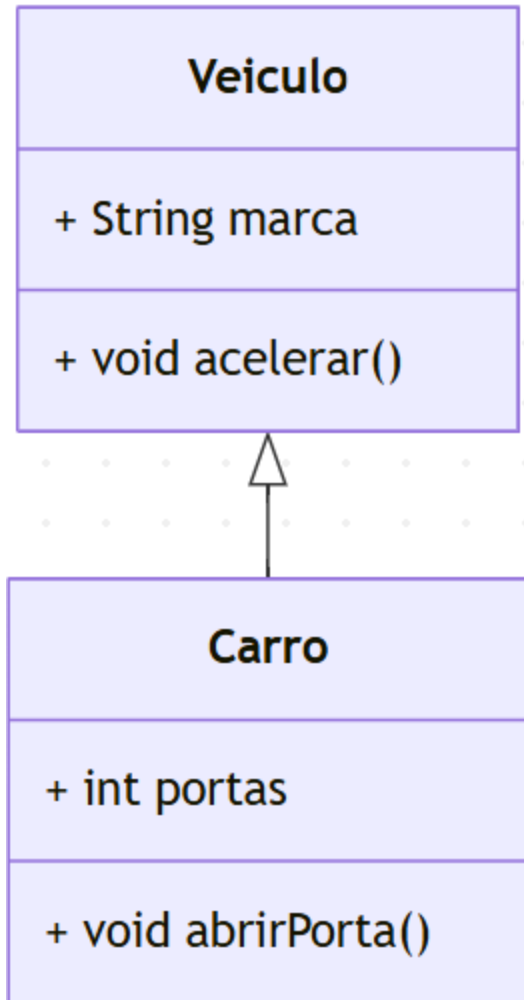
Permite que uma classe herde atributos e métodos de outra, promovendo reuso e hierarquia.

Um objeto, dito filho, herdará características e comportamentos do objeto dito pai.

Vantagens

1. Reutilização de código
 - i. Métodos e atributos da classe base são herdados sem reescrita.
2. Hierarquia e organização
 - i. Modelagem natural de relações "é-um" (ex: Carro é um Veículo).
3. Extensibilidade
 - i. Novas funcionalidades podem ser adicionadas em subclasses sem modificar a superclasse.
4. Polimorfismo
 - i. Permite tratar objetos de subclasses como instâncias da superclasse.

Exemplo



3.3. Polimorfismo

Capacidade de um objeto ser referenciado de múltiplas formas e executar comportamentos diferentes com base no tipo real.

Por meio do polimorfismo, um objeto pode ser referenciado de várias formas, o que confere flexibilidade ao desenvolvimento da solução.

Vantagens

1. Flexibilidade

- i. Um mesmo método pode ter comportamentos diferentes em subclasses (sobrescrita).

2. Código genérico

- i. Tratar objetos de diferentes tipos de forma uniforme (ex: lista de Animal com Cachorro e Gato).

3. Extensibilidade sem modificação

- i. Novas subclasses podem ser adicionadas sem alterar código existente (ex: novo TipoDePagamento em um sistema).

4. Redução de estruturas condicionais

- i. Substitui if/switch por dispatch dinâmico de métodos.

Exemplo

```
Animal animal = new Cachorro(); // Polimorfismo  
animal.emitirSom(); // "Latido"
```

Tipos

- **Sobrescrita (Override):** Mesma assinatura, implementação diferente.
- **Sobrecarga (Overload):** Mesmo nome, parâmetros diferentes.

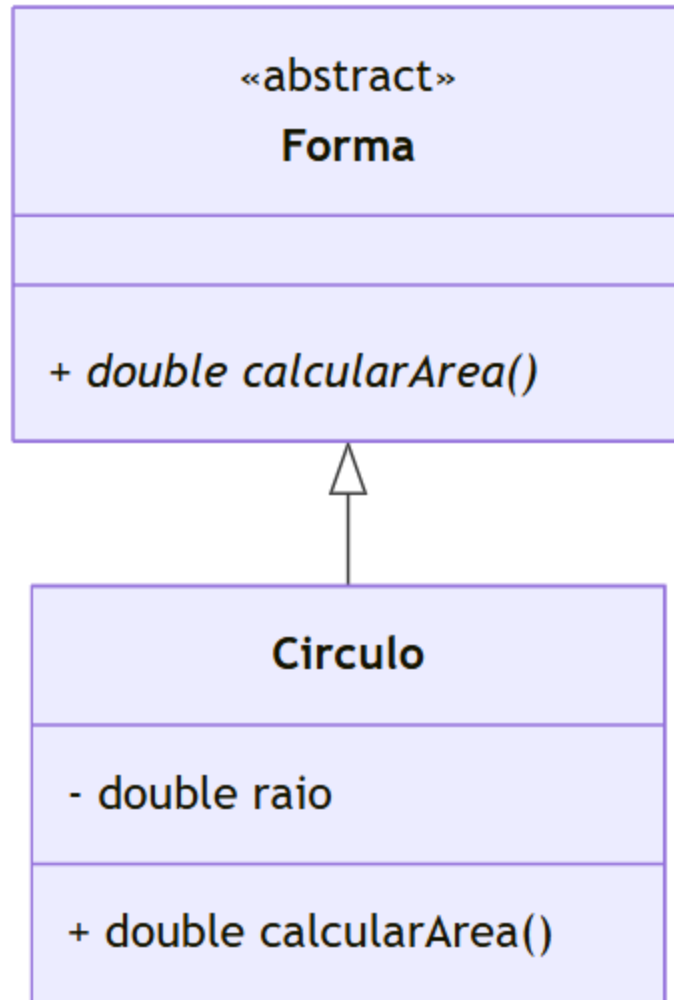
3.4. Abstração

Foca nos aspectos essenciais de um objeto, ignorando detalhes irrelevantes.

Vantagens

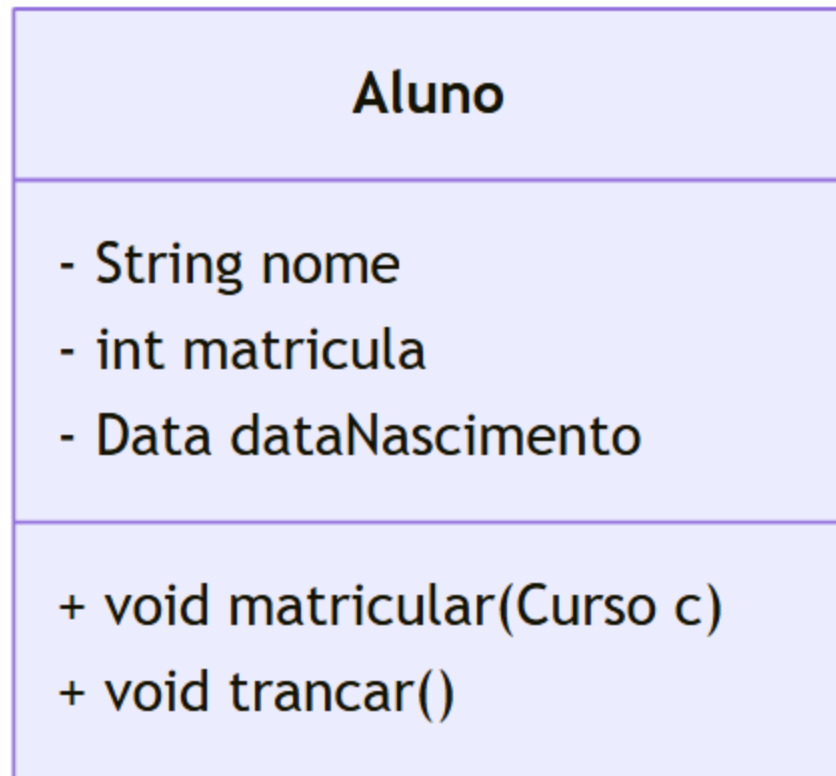
1. Foco no essencial
 - i. Esconde detalhes de implementação irrelevantes para o usuário da classe.
2. Definição de contratos
 - i. Interfaces e classes abstratas definem o que deve ser feito, não como.
3. Desacoplamento
 - i. Classes dependem de abstrações (interfaces), não de implementações concretas.
4. Facilita testes e mudanças
 - i. Troca de implementações (ex: MySQLDatabase → PostgreSQLDatabase).

Exemplo



4. Exemplo Prático: Modelando um Sistema Acadêmico

Classe Aluno



Classe **Curso**

Curso

- String nome
- int codigo
- List<Disciplina> disciplinas

+ void adicionarDisciplina(Disciplina d)

5. Relação entre Classes, Objetos e Instâncias

- **Classe:** Molde (ex: `class Carro`).
- **Objeto:** Instância concreta (ex: `Carro meuCarro = new Carro()`).
- **Referência:** Variável que aponta para o objeto.

6. Exercício Proposto

Modelagem de uma classe `Livro` com:

- Atributos: título, autor, ISBN, emprestado.
- Métodos: `emprestar()`, `devolver()`, `estaEmprestado()`.

Modelo Construído

Livro
- String titulo - String autor - String ISBN - boolean emprestado
+ void emprestar() + void devolver() + boolean estaEmprestado()

7. Leituras Recomendadas

Capítulo 1: Introdução à Programação Orientada a Objetos.

Resumo Técnico

Conceito	Definição
Classe	Molde que define atributos e métodos.
Objeto	Instância de uma classe.
Encapsulamento	Ocultação de dados com acesso controlado.
Herança	Relação "é-um" entre classes.
Polimorfismo	Mesma interface, comportamentos diferentes.
Abstração	Foco no essencial, ignorando detalhes.

Mermaid Example